CHAPTER 25

Typestate: Protocols in Types

Most interfaces come with rules about order. Connect before you log in; log in before you query; calibrate an instrument before you start a measurement, and stop the measurement before you power it down; set the URL of a request before you send it. Such rules form a **protocol**, and usually they live in the documentation. The code enforces them at run time, if at all: a `state` field, a check at the top of every method, an error such as `NotConnected` that someone has to handle and some test has to provoke.

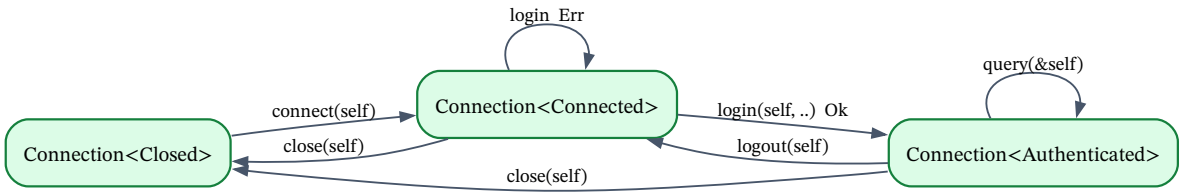
Typestate moves the protocol into the types. Each state of the protocol becomes a type; each operation becomes a function from the type of the state where it is allowed to the type of the state it leads to. Calling an operation in the wrong state is then not a run-time error but a type error, found by the compiler before the program runs. In Rust, the pattern works because of the property that Part V has been studying: values are affine, so a transition can *consume* the old state, and the old state cannot be used again. In this chapter we build typestate APIs in Rust for a database connection, a request builder and a lab instrument; we look briefly at session types, typestate for communication channels; we meet the limit that Chapter 24 predicted (an affine language cannot force a protocol to be completed); and in Lean we use dependent types to go further, with families of types indexed by states, a vault whose type records its balance, and proofs about the traces a protocol accepts.

In this chapter you will learn:

- how to turn a protocol (a state machine) into types, and why consuming methods and affinity are what make the old state unusable;
- the Rust toolkit for typestate: states as type parameters, `PhantomData`, sealed traits for closed sets of states, states that carry data, fallible transitions;
- how to build a builder whose required fields are checked at compile time;
- what session types are, what duality means, and how to encode them in Rust;
- where affinity falls short (nothing forces a protocol to finish), and what `#[must_use]` and a complaining destructor can and cannot do about it;
- how Lean expresses typestate with indexed families, including states that depend on run-time values;
- how to describe a protocol as an inductive relation in Lean and prove that well-typed programs follow it, finish in the final state and stop every measurement they start.

25.1 Protocols as state machines

A protocol is a *labelled transition system*: a set of states, a set of actions, and a transition relation saying which action is allowed in which state and where it leads. Here is the protocol of the database connection we will implement:



The connection protocol. Each box is a state, which will be a type; each arrow is an operation. Operations that take self by value consume the old state and return the new one; query takes &self and stays where it is.

Writing $\delta(s, a) = s'$ for "in state s, action a is allowed and leads to s'", the diagram says $\delta(\mathit{Closed}, \texttt{connect}) = \mathit{Connected}$, $\delta(\mathit{Authenticated}, \texttt{query}) = \mathit{Authenticated}$, and so on; δ is undefined everywhere else.

The run-time approach stores the current state in a field and checks it:

```
fn query(&self, sql: &str) -> Result<Rows, ConnError> {
    if self.state != State::Authenticated {
        return Err(ConnError::NotAuthenticated);
    }
    // ...
}
```

The typestate approach gives the connection a type parameter S for its state, and makes each action a method that exists only for the right S. As a typing rule (a *model* of what the Rust code below does, in the sense of Chapter 1):

$$\frac{\Gamma \vdash c : \texttt{Connection}\langle s \rangle \qquad \delta(s, a) = s'}{\Gamma \vdash c.a() : \texttt{Connection}\langle s' \rangle}\ \text{T-Transition}$$

The second premise is not a judgment the compiler derives; it is a fact built into the program by writing an `impl` block for each defined $\delta(s, a)$. If $\delta(s, a)$ is undefined, no rule applies, and the compiler reports that the method does not exist for that type.

The idea is old. Robert Strom and Shaula Yemini proposed typestate in the 1980s as a compiler analysis, and early, pre-1.0 versions of Rust had a typestate system built into the language, later removed. What Rust programmers call the typestate pattern today needs no special support: it is built from generics, traits and ownership.

Why affinity matters

The rule T-Transition has a hidden assumption. After `let c2 = c.connect();`, what happens to `c`? If `c` can still be used, the program has two views of the same connection, one saying `Closed` and one saying `Connected`, and at most one of them is telling the truth. The type of `c` would be a lie.

In a language with contraction, nothing stops you from using `c` again. In Rust, `connect` takes `self` by value, so the call *moves* `c`, and every later use is rejected (E0382). That is the affine discipline of Chapter 22 doing exactly the job we need: the old state is consumed by the transition, and only the new state remains. Here is the error for using a connection after closing it, from `code/rust/compile_fail/ch25_use_after_close.rs`:

```
pub fn finish(mut conn: Connection<Connected>) -> Connection<Closed> {
    let closed = conn.close();
    conn.send("QUIT");
    closed
}
```

```
error[E0382]: borrow of moved value: `conn`
  --> compile_fail/ch25_use_after_close.rs:31:5
   |
29 | pub fn finish(mut conn: Connection<Connected>) -> Connection<Closed> {
   |               -------- move occurs because `conn` has type `Connection<Connected>`,
which does not implement the `Copy` trait
30 |     let closed = conn.close();
   |                       ------- `conn` moved due to this method call
31 |     conn.send("QUIT");
   |     ^^^^ value borrowed here after move
   |
note: `Connection::<Connected>::close` takes ownership of the receiver `self`, which moves
`conn`
  --> compile_fail/ch25_use_after_close.rs:20:18
   |
20 |     pub fn close(self) -> Connection<Closed> {
   |                  ^^^^
```

The note at the end names the culprit precisely: `close` takes ownership of the receiver `self`. In typestate terms, `close` consumed the `Connection<Connected>`, and the only connection left is the `Connection<Closed>` it returned.

25.2 Typestate in Rust: a connection

The full program is `code/rust/examples/ch25_typestate.rs`. It contains two modules, `connection` and `instrument`, and a `main` that uses them.

The states

```
mod sealed {
    /// Only this module can implement `Sealed`, so only the three states
    /// below can ever implement `State`.
    pub trait Sealed {}
}

/// The states of the protocol: a closed set of types.
pub trait State: sealed::Sealed {
    const NAME: &'static str;
}

/// States in which the connection is open.
pub trait Open: State {}

// Empty enums: types with no values, used only as labels.
pub enum Closed {}
pub enum Connected {}
pub enum Authenticated {}
```

```
impl sealed::Sealed for Closed {}
impl sealed::Sealed for Connected {}
impl sealed::Sealed for Authenticated {}
impl State for Closed {
    const NAME: &'static str = "Closed";
}
impl State for Connected {
    const NAME: &'static str = "Connected";
}
impl State for Authenticated {
    const NAME: &'static str = "Authenticated";
}
impl Open for Connected {}
impl Open for Authenticated {}
```

Each state is a type with *no values*: an empty enum, like the `Void` of Chapters 5 and 10. We never create a `Closed`; we only use the type as a label, which is why an empty enum is the right choice (a unit struct would work too, but it would suggest that values of it exist). The trait `State` gathers the three labels and gives each a name for printing.

`State` is **sealed**. Its supertrait `sealed::Sealed` lives in a private module, so code outside `connection` can *use* `State` but cannot *implement* it: no one else can add a fourth state, and the set of states is closed, like the variants of an enum. Without the seal, a user could implement `State` for their own type and create a `Connection<TheirState>` that the protocol knows nothing about. With it, the attempt fails (`code/rust/compile_fail/ch25_sealed.rs`):

```
pub struct Hijacked;

impl connection::State for Hijacked {}
```

```
error[E0277]: the trait bound `Hijacked: Sealed` is not satisfied
  --> compile_fail/ch25_sealed.rs:19:28
   |
19 | impl connection::State for Hijacked {}
   |                            ^^^^^^^^ unsatisfied trait bound
   |
help: the trait `Sealed` is not implemented for `Hijacked`
  --> compile_fail/ch25_sealed.rs:17:1
   |
17 | pub struct Hijacked;
   | ^^^^^^^^^^^^^^^^^^^
help: the trait `Sealed` is implemented for `Closed`
  --> compile_fail/ch25_sealed.rs:12:5
   |
12 |     impl sealed::Sealed for Closed {}
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: required by a bound in `State`
  --> compile_fail/ch25_sealed.rs:9:22
   |
 9 |     pub trait State: sealed::Sealed {}
   |                      ^^^^^^^^^^^^^^ required by this bound in `State`
   = note: `State` is a "sealed trait", because to implement it you also need to implement
`connection::sealed::Sealed`, which is not accessible; this is usually done to force you to
use one of the provided types that already implement it
   = help: the following type implements the trait:
             connection::Closed
```

The compiler's last note even names the pattern. A second trait, `Open`, describes a *subset* of the states, the two in which the connection is open; we will use it to define `close` once for both.

The connection type

```
/// A connection in state `S`. The state exists only in the type:
/// `PhantomData<S>` has size zero.
#[must_use = "transitions return the connection in its new state"]
pub struct Connection<S: State> {
    host: String,
    _state: PhantomData<S>,
}

impl<S: State> Connection<S> {
    /// Available in every state.
    pub fn describe(&self) -> String {
        format!("{} [{}]", self.host, S::NAME)
    }

    /// Change the state: same data, new type.
    fn into_state<T: State>(self) -> Connection<T> {
        Connection {
            host: self.host,
            _state: PhantomData,
        }
    }
}
```

The state appears only in the type. The connection's data, a host name, is the same in every state, and the field `_state: PhantomData<S>` exists only to mention `S`. Rust insists that every type parameter of a struct be used by some field, because it needs to know how the struct relates to the parameter (Chapter 27 explains the *variance* it infers from the fields):

```
pub struct Connection<S> {
    host: String,
}
```

```
error[E0392]: type parameter `S` is never used
 --> compile_fail/ch25_unused_state.rs:6:23
  |
6 | pub struct Connection<S> {
  |                       ^ unused type parameter
  |
  = help: consider removing `S`, referring to it in a field, or using a marker such as
`PhantomData`
  = help: if you intended `S` to be a const parameter, use `const S: /* Type */` instead
```

`PhantomData<S>` is the standard answer: a zero-sized field that tells the compiler "treat this struct as if it contained an `S`". It costs nothing at run time, as the last line of the program's output will confirm.

The `impl<S: State>` block defines methods that exist in *every* state: `describe`, which prints the host and the state's name, and the private helper `into_state`, which rebuilds the same data with a different state type. Every transition will be a call to `into_state`. The attribute `#[must_use]` comes back later in the chapter.

The transitions

```
impl Connection<Closed> {
    pub fn new(host: &str) -> Self {
        Connection {
            host: host.to_string(),
            _state: PhantomData,
        }
    }
```

```
    pub fn connect(self) -> Connection<Connected> {
        self.into_state()
    }
}

impl Connection<Connected> {
    /// A fallible transition: on failure, the caller gets the connection
    /// back, still in state `Connected`.
    pub fn login(
        self,
        user: &str,
        password: &str,
    ) -> Result<Connection<Authenticated>, Connection<Connected>> {
        if user == "lab" && password == "secret" {
            Ok(self.into_state())
        } else {
            Err(self)
        }
    }
}

impl Connection<Authenticated> {
    /// Does not change the state: takes `&self`, not `self`.
    pub fn query(&self, sql: &str) -> String {
        format!("result of `{sql}` from {}", self.host)
    }

    pub fn logout(self) -> Connection<Connected> {
        self.into_state()
    }
}

impl<S: Open> Connection<S> {
    /// Close from any open state.
    pub fn close(self) -> Connection<Closed> {
        self.into_state()
    }
}
```

Each `impl` block is one row of the transition table. `impl Connection<Closed>` defines `new` and `connect`, which exist only for closed connections; `impl Connection<Authenticated>` defines `query` and `logout`; and `impl<S: Open> Connection<S>` defines `close` once for every open state. Three details are worth a closer look.

- **Transitions take `self`, observations take `&self`.** `connect`, `login`, `logout` and `close` consume the connection and return it at a new type. `query` borrows it and leaves the state unchanged, which is why it is a loop in the diagram.
- **Fallible transitions return both outcomes.** A login can fail. `login` returns `Result<Connection<Authenticated>, Connection<Connected>>`: on success the authenticated connection, on failure the *original* connection, still connected, so the caller can try again. Since `login` consumed `self`, returning it in the error case is the only way to give it back. When a transition depends on run-time information, the type cannot know the outcome, and an enum of possible states (`Result` here) is how the uncertainty shows up in the types.
- **Same data, different types.** Every state has the same representation, so `into_state` just moves the host name into a struct of a different type. The compiler sees `Connection<Closed>` and `Connection<Connected>` as two unrelated types; at run time they are the same bytes.

Using it

```
fn session() {
    let conn = Connection::new("db.lab.internal");
    println!("  {}", conn.describe());
    let conn = conn.connect();
    println!("  {}", conn.describe());
    let conn = match conn.login("lab", "wrong") {
        Ok(_) => unreachable!("the password is wrong"),
        Err(conn) => {
            println!("  login failed, still {}", conn.describe());
            conn
        }
    };
    let Ok(conn) = conn.login("lab", "secret") else {
        unreachable!("the password is right");
    };
    println!("  {}", conn.describe());
    println!("  {}", conn.query("SELECT * FROM samples"));
    let conn = conn.logout().close();
    println!("  {}", conn.describe());
}
```

```
connection:
  db.lab.internal [Closed]
  db.lab.internal [Connected]
  login failed, still db.lab.internal [Connected]
  db.lab.internal [Authenticated]
  result of `SELECT * FROM samples` from db.lab.internal
  db.lab.internal [Closed]
```

The code reads like the protocol diagram. Each `let conn = ...` shadows the previous `conn` with a value of a new type: the old one was consumed by the transition, so there is nothing to confuse it with. The failed login hands back the connection, still `Connected`, and the second attempt succeeds. `let ... else` destructures the `Ok` case, and otherwise runs the `else` block, which must diverge (here with `unreachable!`).

Now break the protocol. Query before logging in (`code/rust/compile_fail/ch25_wrong_state.rs`):

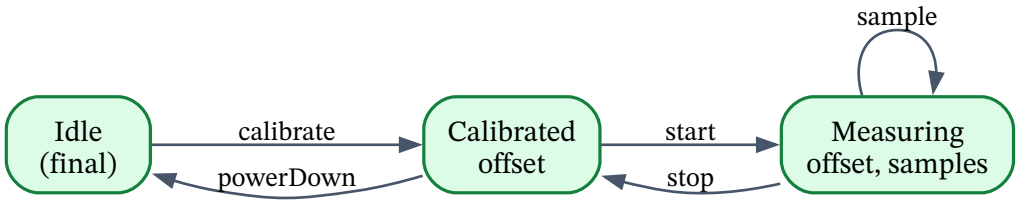
```
pub fn report(conn: Connection<Connected>) -> String {
    conn.query("SELECT * FROM samples")
}
```

```
error[E0599]: no method named `query` found for struct `Connection<Connected>` in the
current scope
  --> compile_fail/ch25_wrong_state.rs:32:10
   |
10 | pub struct Connection<S> {
   | ------------------------ method `query` not found for this struct
...
32 |     conn.query("SELECT * FROM samples")
   |          ^^^^^ method not found in `Connection<Connected>`
   |
   = note: the method was found for `Connection<Authenticated>`
```

This is E0599, "no method found", and it is exactly rule T-Transition failing: $\delta(\mathit{Connected}, \texttt{query})$ is undefined, so there is no `query` for `Connection<Connected>`. The note, "the method was found for `Connection<Authenticated>`", is the compiler pointing at the state where the operation *is* allowed. A protocol violation has become a type error with a good error message.

25.3 States that carry data: a lab instrument

In the connection, every state had the same data. Often the states differ in what is *known*. A lab instrument starts `Idle`. Calibrating it against a reference gives an offset, which every later reading must be corrected by. While `Measuring`, it accumulates samples. A run-time design would put `offset: Option<f64>` and `samples: Vec<f64>` in one struct and `unwrap` the offset wherever it is needed, hoping it was set. With typestate, each state holds exactly the data that exists in it:



The lab instrument protocol. Calibrated knows its offset; Measuring knows its offset and its samples. Idle is the final state: a session should end there.

```
/// The states carry data: what is known in that state, and nothing more.
pub struct Idle;
pub struct Calibrated {
    offset: f64,
}
pub struct Measuring {
    offset: f64,
    samples: Vec<f64>,
    armed: bool,
}

pub struct Instrument<S> {
    name: String,
    state: S,
}
```

Here the type parameter `S` *is* the type of a real field, `state: S`, so no `PhantomData` is needed. A `Calibrated` contains an offset, so an `Instrument<Calibrated>` without an offset cannot be constructed; the `unwrap` and the bug it might hide are gone. This is the slogan "make illegal states unrepresentable" applied to protocols. The transitions move data from state to state:

```
impl Instrument<Idle> {
    pub fn new(name: &str) -> Self {
        Instrument {
            name: name.to_string(),
            state: Idle,
        }
    }

    /// Compare a reading of a known reference with its true value.
    pub fn calibrate(self, reference: f64, reading: f64) -> Instrument<Calibrated> {
        let offset = reference - reading;
        Instrument {
            name: self.name,
            state: Calibrated { offset },
        }
    }
}
```

```
impl Instrument<Calibrated> {
    pub fn start(self) -> Instrument<Measuring> {
        let offset = self.state.offset;
        let state = Measuring {
            offset,
            samples: Vec::new(),
            armed: true,
        };
        Instrument {
            name: self.name,
            state,
        }
    }

    pub fn power_down(self) -> Instrument<Idle> {
        Instrument {
            name: self.name,
            state: Idle,
        }
    }
}

impl Instrument<Measuring> {
    pub fn record(&mut self, raw: f64) {
        self.state.samples.push(raw + self.state.offset);
    }

    /// Finish the measurement: back to `Calibrated`, with the samples.
    pub fn stop(mut self) -> (Instrument<Calibrated>, Vec<f64>) {
        // `Measuring` implements `Drop`, so its fields cannot be moved out
        // (E0509): take the samples, and disarm the complaint.
        let samples = std::mem::take(&mut self.state.samples);
        self.state.armed = false;
        let state = Calibrated {
            offset: self.state.offset,
        };
        (
            Instrument {
                name: self.name,
                state,
            },
            samples,
        )
    }
}

impl<S> Instrument<S> {
    pub fn name(&self) -> &str {
        &self.name
    }
}
```

`calibrate` computes the offset and stores it in the new state. `start` copies the offset into a fresh `Measuring`. `record` takes `&mut self`: recording a sample changes the data but not the state. `stop` returns the instrument in state `Calibrated` together with the samples. Its body is more complicated than you might expect, because of a destructor we will meet in the section on what affinity cannot do. Here is the program using it:

```
fn measure() {
    let probe = Instrument::new("thermometer").calibrate(100.0, 99.5);
    let mut run = probe.start();
    for raw in [20.0, 20.5, 21.0] {
        run.record(raw);
    }
    let (probe, samples) = run.stop();
    println!("  {}: samples {samples:?}", probe.name());
```

```
    let probe = probe.power_down().calibrate(100.0, 100.0);
    let mut run = probe.start();
    run.record(22.0);
    run.record(22.5);
    println!("  dropping {} in the middle of a measurement", run.name());
} // `run` is dropped here, still in state Measuring
```

```
instrument:
  thermometer: samples [20.5, 21.0, 21.5]
  dropping thermometer in the middle of a measurement
  warning: measurement abandoned, 2 sample(s) lost
size of Connection<Authenticated>: 24 bytes (a String)
```

The first measurement is complete: the probe read 99.5 for a reference of 100, so the offset is 0.5, and the three raw readings come back corrected. The second measurement is *abandoned*: the function ends while `run` is still `Instrument<Measuring>`. Rust accepts this, because a value may always be dropped, and the only trace of the mistake is the warning printed by a destructor. That line is the subject of a later section. The last line of output shows the cost of the typestate machinery in the connection: `Connection<Authenticated>` is 24 bytes, the size of its `String`, and the state type adds nothing.

25.4 Builders with required fields

The builder pattern constructs a complicated value step by step: `Request::builder().method(...).url(...).header(...).build()`. Some fields are required, and the usual implementation checks them in `build` (`code/rust/examples/ch25_builder.rs`):

```
/// The usual builder: required fields are `Option`s, checked by `build`.
#[derive(Default)]
pub struct DynBuilder {
    method: Option<Method>,
    url: Option<String>,
    timeout_ms: Option<u64>,
}

#[derive(Debug)]
pub enum BuildError {
    MissingMethod,
    MissingUrl,
}

impl DynBuilder {
    pub fn method(mut self, method: Method) -> Self {
        self.method = Some(method);
        self
    }

    pub fn url(mut self, url: &str) -> Self {
        self.url = Some(url.to_string());
        self
    }

    pub fn build(self) -> Result<Request, BuildError> {
        Ok(Request {
            method: self.method.ok_or(BuildError::MissingMethod)?,
            url: self.url.ok_or(BuildError::MissingUrl)?,
            headers: Vec::new(),
            timeout_ms: self.timeout_ms.unwrap_or(30_000),
        })
```

```
    }
}
```

```
let missing = DynBuilder::default().method(Method::Get).build();
println!("dynamic, URL forgotten: {missing:?}");
let ok = DynBuilder::default()
    .url("/samples")
    .method(Method::Get)
    .build();
println!("dynamic, complete:      {}", ok.expect("both fields set"));
```

```
dynamic, URL forgotten: Err(MissingUrl)
dynamic, complete:      Get /samples (timeout 30000 ms)
```

Forgetting the URL is an error at run time, in a `Result` that every caller must handle even when, as in most calls, every field was set. A builder is a protocol too ("set the method and the URL, in any order, then build"), and typestate can check it. The state of the builder is "which required fields are set", one type parameter per field:

```
/// Marker types: "not set yet". The set versions carry the value itself.
pub struct NoMethod;
pub struct NoUrl;
pub struct Url(String);

/// `M` is `NoMethod` or `Method`; `U` is `NoUrl` or `Url`. No `PhantomData`
/// is needed: the type parameters are the types of real fields.
pub struct RequestBuilder<M, U> {
    method: M,
    url: U,
    headers: Vec<(String, String)>,
    timeout_ms: u64,
}

impl RequestBuilder<NoMethod, NoUrl> {
    pub fn new() -> Self {
        RequestBuilder {
            method: NoMethod,
            url: NoUrl,
            headers: Vec::new(),
            timeout_ms: 30_000,
        }
    }
}

/// Setting the method: only possible while it is not set, whatever `U` is.
impl<U> RequestBuilder<NoMethod, U> {
    pub fn method(self, method: Method) -> RequestBuilder<Method, U> {
        RequestBuilder {
            method,
            url: self.url,
            headers: self.headers,
            timeout_ms: self.timeout_ms,
        }
    }
}

/// Setting the URL: only possible while it is not set, whatever `M` is.
impl<M> RequestBuilder<M, NoUrl> {
    pub fn url(self, url: &str) -> RequestBuilder<M, Url> {
        RequestBuilder {
            method: self.method,
            url: Url(url.to_string()),
            headers: self.headers,
            timeout_ms: self.timeout_ms,
```

```
        }
    }
}

/// Optional settings: available in every state, and they keep the state.
impl<M, U> RequestBuilder<M, U> {
    pub fn header(mut self, name: &str, value: &str) -> Self {
        self.headers.push((name.to_string(), value.to_string()));
        self
    }

    pub fn timeout_ms(mut self, ms: u64) -> Self {
        self.timeout_ms = ms;
        self
    }
}

/// `build` exists only when both required fields are set.
impl RequestBuilder<Method, Url> {
    pub fn build(self) -> Request {
        Request {
            method: self.method,
            url: self.url.0,
            headers: self.headers,
            timeout_ms: self.timeout_ms,
        }
    }
}
```

The type parameter `M` is `NoMethod` until a method is set, and then it is `Method`, the type of the method itself; likewise `U` is `NoUrl` or `Url`. The markers for "not set" are unit structs, and the "set" types hold the value, so again every parameter is the type of a real field and no `PhantomData` is needed. Read the `impl` blocks as transitions:

- `impl<U> RequestBuilder<NoMethod, U>` defines `method` for every builder whose method is not set yet, *whatever* the state of the URL. That generic `U` is what makes the order free: the method can be set before or after the URL.
- `impl<M> RequestBuilder<M, NoUrl>` does the same for `url`.
- `impl<M, U> RequestBuilder<M, U>` defines the optional settings for every state; they return `Self`, the same type, so they do not change the state.
- `impl RequestBuilder<Method, Url>` defines `build`, only when both are set. It returns a `Request`, not a `Result`: there is nothing left to go wrong.

```
let a = RequestBuilder::new()
    .method(Method::Post)
    .header("content-type", "application/json")
    .url("/samples")
    .timeout_ms(500)
    .build();
let b = RequestBuilder::new()
    .url("/samples/42")
    .method(Method::Get)
    .build();
println!("typestate, method first: {a}");
println!("typestate, URL first:    {b}");
```

```
typestate, method first: Post /samples (timeout 500 ms, content-type: application/json)
typestate, URL first:    Get /samples/42 (timeout 30000 ms)
size of RequestBuilder<NoMethod, NoUrl>: 32 bytes
size of RequestBuilder<Method, Url>:     64 bytes
```

Both orders work, optional settings can appear anywhere in the chain, and the two sizes show that the type of the builder changes as it fills up. A missing field is now a compile-time error (`code/rust/compile_fail/ch25_builder_incomplete.rs`):

```
pub fn request() -> (String, String) {
    RequestBuilder::new().method("GET").build()
}
```

```
error[E0599]: no method named `build` found for struct `RequestBuilder<Method, NoUrl>` in
the current scope
  --> compile_fail/ch25_builder_incomplete.rs:41:41
   |
10 | pub struct RequestBuilder<M, U> {
   | ------------------------------- method `build` not found for this struct
...
41 |     RequestBuilder::new().method("GET").build()
   |                                         ^^^^^ method not found in
`RequestBuilder<Method, NoUrl>`
   |
   = note: the method was found for `RequestBuilder<Method, Url>`
```

The type in the message, `RequestBuilder<Method, NoUrl>`, tells you which field is missing. Setting the method twice is rejected the same way, since `method` exists only while `M` is `NoMethod`.

The technique has costs, and they grow with the number of required fields. Each field adds a type parameter and doubles the number of possible builder types; each transition method has to move every field into the new struct (the field lists in `method` and `url`); and the error messages mention types that the user never wrote. For two or three required fields it is usually worth it; for ten, crates that generate typestate builders with a derive macro exist, and a run-time check may also be the more readable choice.

25.5 Session types, briefly

The protocols so far govern one object. A **session type** governs a *conversation*: the type of a communication channel describes the whole sequence of messages that will travel on it. Session types were introduced by Kohei Honda and his colleagues in the 1990s. A small syntax:

$$S ::= !\tau.\, S \mid ?\,\tau.\, S \mid \mathit{end}$$

Here $!\tau.\, S$ means "send a value of type τ, then continue as S", $?\,\tau.\, S$ means "receive a τ, then continue as S", and *end* means that the conversation is over. A client that sends a sensor number and receives a reading has type $!\texttt{u32}.\, ?\,\texttt{f64}.\, \mathit{end}$.

The server at the other end of the channel must do the opposite: receive where the client sends, send where the client receives. This mirror image is the **dual** of a session type:

$$\overline{!\tau.\, S} = ?\,\tau.\, \overline{S} \qquad \overline{?\,\tau.\, S} = !\tau.\, \overline{S} \qquad \overline{\mathit{end}} = \mathit{end}$$

Duality is the central idea: if the two ends of a channel have dual types, every send is matched by a receive of the same type, and neither side waits for a message the other will never send. Duality is also an *involution*: the dual of the dual is the original, $\overline{\overline{S}} = S$. (If you know linear logic, this is the duality of linear negation: session types can be read as propositions of linear logic, the logic of Chapter 21.)

In Rust

The example `code/rust/examples/ch25_session.rs` encodes session types as types, and duality as a trait with an associated type:

```rust
/// "Send a `T`, then continue as `S`."
pub struct Send<T, S>(PhantomData<(T, S)>);
/// "Receive a `T`, then continue as `S`."
pub struct Recv<T, S>(PhantomData<(T, S)>);
/// "The conversation is over."
pub struct End;

/// Every session type has a dual: the same protocol seen from the other side.
pub trait Session {
    type Dual: Session;
}

impl Session for End {
    type Dual = End;
}
impl<T, S: Session> Session for Send<T, S> {
    type Dual = Recv<T, S::Dual>;
}
impl<T, S: Session> Session for Recv<T, S> {
    type Dual = Send<T, S::Dual>;
}
```

The three `impl` blocks are the three equations of duality, computed by the trait solver at compile time. A protocol is a type, and its dual is computed, not written:

```rust
/// The client names a sensor, receives its reading, sends back a correction
/// and learns whether the corrected value is in range.
type Client = Send<u32, Recv<f64, Send<f64, Recv<bool, End>>>>;
/// The server's protocol is computed, not written by hand.
type Server = <Client as Session>::Dual;

/// Compiles only if the dual of `A` is `B`.
fn assert_dual<A: Session<Dual = B>, B>() {}
```

`assert_dual::<A, B>()` compiles only if `B` is the dual of `A`, so the two calls in `main` make the compiler check $\overline{Client} = Server$ and $\overline{Server} = Client$. A channel endpoint is a typestate object whose state is the rest of the protocol:

```rust
/// What travels on the wire.
#[derive(Debug)]
enum Msg {
    Nat(u32),
    Real(f64),
    Flag(bool),
}

/// The payload types the wire can carry.
trait Payload: Sized {
    fn wrap(self) -> Msg;
    fn unwrap(msg: Msg) -> Option<Self>;
}

impl Payload for u32 {
    fn wrap(self) -> Msg {
        Msg::Nat(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Nat(n) = msg { Some(n) } else { None }
    }
}
```

```
impl Payload for f64 {
    fn wrap(self) -> Msg {
        Msg::Real(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Real(x) = msg {
            Some(x)
        } else {
            None
        }
    }
}

impl Payload for bool {
    fn wrap(self) -> Msg {
        Msg::Flag(self)
    }
    fn unwrap(msg: Msg) -> Option<Self> {
        if let Msg::Flag(b) = msg {
            Some(b)
        } else {
            None
        }
    }
}

type Queue = Rc<RefCell<VecDeque<Msg>>>;

/// An endpoint that must follow protocol `S`. Every operation consumes the
/// endpoint and returns it at the next step of the protocol.
struct Chan<S> {
    incoming: Queue,
    outgoing: Queue,
    _protocol: PhantomData<S>,
}

impl<S: Session> Chan<S> {
    /// Two connected endpoints, with dual protocols.
    fn pair() -> (Chan<S>, Chan<S::Dual>) {
        let a_to_b: Queue = Rc::default();
        let b_to_a: Queue = Rc::default();
        let a = Chan {
            incoming: Rc::clone(&b_to_a),
            outgoing: Rc::clone(&a_to_b),
            _protocol: PhantomData,
        };
        let b = Chan {
            incoming: a_to_b,
            outgoing: b_to_a,
            _protocol: PhantomData,
        };
        (a, b)
    }
}

impl<S> Chan<S> {
    fn advance<S2>(self) -> Chan<S2> {
        Chan {
            incoming: self.incoming,
            outgoing: self.outgoing,
            _protocol: PhantomData,
        }
    }
}

impl<T: Payload, S> Chan<Send<T, S>> {
    fn send(self, value: T) -> Chan<S> {
        self.outgoing.borrow_mut().push_back(value.wrap());
        self.advance()
```

```
    }
}

impl<T: Payload, S> Chan<Recv<T, S>> {
    fn recv(self) -> (T, Chan<S>) {
        let msg = self.incoming.borrow_mut().pop_front();
        let value = msg
            .and_then(T::unwrap)
            .expect("the peer follows the dual protocol");
        (value, self.advance())
    }
}

impl Chan<End> {
    fn close(self) {}
}
```

The parts to read are the four `impl` blocks at the end. `send` exists only on a `Chan<Send<T, S>>`, takes a `T` and returns a `Chan<S>`: one step of the protocol consumed. `recv` exists only on a `Chan<Recv<T, S>>` and returns the received value with a `Chan<S>`. `close` exists only on a `Chan<End>`. And `pair` creates two connected endpoints whose types are dual *by construction*: `Chan<S>` and `Chan<S::Dual>`. (The queues and the `Payload` trait are plumbing for an in-memory wire; a real library would use threads or an async runtime.)

```
assert_dual::<Client, Server>();
assert_dual::<Server, Client>();
let short = |name: &str| name.replace("ch25_session::", "");
println!("Client = {}", short(std::any::type_name::<Client>()));
println!("Server = {}", short(std::any::type_name::<Server>()));

// The two sides take turns (in a real program, two threads or tasks).
let (client, server) = Chan::<Client>::pair();
let client = client.send(7_u32);
let (sensor, server) = server.recv();
let server = server.send(21.5_f64);
let (reading, client) = client.recv();
let client = client.send(reading - 0.5);
let (corrected, server) = server.recv();
let server = server.send((15.0..=25.0).contains(&corrected));
let (in_range, client) = client.recv();
client.close();
server.close();
println!("sensor {sensor}: reading {reading}, corrected {corrected}");
println!("in range: {in_range}");
```

```
Client = Send<u32, Recv<f64, Send<f64, Recv<bool, End>>>>
Server = Recv<u32, Send<f64, Recv<f64, Send<bool, End>>>>
sensor 7: reading 21.5, corrected 21
in range: true
```

The two type names in the output are each other's mirror image. In the conversation, notice what is *not* written: no type annotations on `recv`. When the server calls `server.recv()`, the type of its endpoint is `Chan<Recv<u32, ...>>`, so the compiler knows the value is a `u32`. Calling `client.recv()` first, or sending a `bool` where a `u32` is expected, is a type error.

The encoding has the limits you would now expect. It guarantees "no wrong message": each endpoint can only perform the next step of its protocol. It does not guarantee "no missing message": an endpoint can be dropped (or forgotten) in the middle of the conversation, and the peer then waits for a message that never comes (in our in-memory version, `recv` would panic). Real protocols also need choice

(offer several continuations) and recursion (loops), which make the encodings considerably heavier. The Lean section below shows what a language that can check *totality* adds.

25.6 Where affinity falls short

Typestate turns "operation in the wrong state" into a type error. It does not turn "protocol left unfinished" into one. The second measurement of the lab instrument was abandoned while `Measuring`, and the program compiled; a `Connection` can be dropped while authenticated; a session endpoint can be dropped before `End`. The reason is the one Chapter 24 gave: Rust keeps weakening, so any value may be discarded at any point, and `mem::forget` and `Rc` cycles may even discard it without running its destructor. A protocol that *must* reach a final state needs linear types, which Rust does not have.

What we can do is approximate. There are three common mitigations, each with its own limits.

`#[must_use]`

The `Connection` struct carries `#[must_use = "transitions return the connection in its new state"]`. Since every transition returns a new `Connection`, the most likely way to lose one is to call a transition and ignore its result: `conn.connect();` on its own line. The compiler then warns "unused `Connection` that must be used", followed by the note from the attribute. As we saw in Chapter 24, this is a lint about one expression statement: it does not notice a connection stored in a variable and never used again, and `let _ = ...` silences it.

A destructor that complains

The second mitigation is a run-time check at the point of abandonment: a `Drop` implementation that notices when a value is dropped in a non-final state, and reports it. That is what produced the warning line in the instrument's output:

```
/// A destructor that complains: the protocol says a measurement must be
/// stopped, and the type system cannot force it.
impl Drop for Measuring {
    fn drop(&mut self) {
        if self.armed {
            let n = self.samples.len();
            println!("  warning: measurement abandoned, {n} sample(s) lost");
        }
    }
}
```

There are three things to understand about this small piece of code.

The destructor is on the state, not on the instrument. The natural place for it would be `impl Drop for Instrument<Measuring>`, a destructor for one state only. Rust forbids that (`code/rust/compile_fail/ch25_drop_one_state.rs`):

```
impl Drop for Instrument<Measuring> {
    fn drop(&mut self) {
        println!("{} dropped while measuring", self.name);
    }
}
```

```
error[E0366]: `Drop` impls cannot be specialized
  --> compile_fail/ch25_drop_one_state.rs:14:1
   |
14 | impl Drop for Instrument<Measuring> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: `Measuring` is not a generic parameter
note: use the same sequence of generic lifetime, type and const parameters as the struct
definition
  --> compile_fail/ch25_drop_one_state.rs:8:1
   |
 8 | pub struct Instrument<S> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^
```

A `Drop` impl must be exactly as generic as the type (E0366): either every `Instrument<S>` has the destructor or none does. This is a deliberate restriction: the destructor of a generic type is one piece of generic code, the same for all its instances. Since our states carry data, we can put the destructor on the state type `Measuring` instead, which has no type parameters, and `Instrument<Measuring>` inherits it through its field. (With zero-sized marker states, the workaround is an `impl<S: State> Drop` that looks at a constant of the state, such as `S::NAME`.)

The destructor gets in the way of the transitions. Look again at `stop`. Since `Measuring` implements `Drop`, its fields cannot be moved out (E0509, Chapter 24), so `stop` cannot simply take `self.state.samples`. It uses `mem::take` to swap the samples out for an empty vector, copies the offset (an `f64` is `Copy`), and then sets `armed` to `false` before returning. The old `Measuring` value is dropped at the end of `stop`, as every consumed state is, and the `armed` flag is how the destructor tells "consumed by `stop`" from "abandoned". Every typestate API with a complaining destructor needs some version of this disarming step.

It complains, it does not prevent. The check happens at run time, when the damage is done. It cannot run at all if the value is forgotten. And it should not panic: a destructor that panics while the thread is already unwinding from another panic aborts the process (Chapter 24). Logging, counting or, in debug builds, an assertion are the usual choices.

Keeping control

The strongest mitigation is the one `thread::scope` used in Chapter 24: do not hand the caller a value that must be finished; take a closure, and finish the protocol yourself after the closure returns. For the instrument:

```
impl Instrument<Calibrated> {
    fn measure(
        self,
        f: impl FnOnce(&mut Recorder),
    ) -> (Instrument<Calibrated>, Vec<f64>) {
        let mut run = self.start();
        f(&mut Recorder(&mut run)); // the caller records samples
        run.stop() // and we always stop
    }
}
```

The caller never holds an `Instrument<Measuring>`, so it cannot abandon one (only a panic inside `f` can interrupt the protocol, and then unwinding runs the destructors). Exercise 4 asks you to write it. The price is flexibility: the measurement must fit inside one closure call.

25.7 Typestate in Lean: indexed families

Lean has no ownership and no affinity, but it has something Rust lacks: types that depend on values. That changes what typestate looks like. The Lean files of this chapter are in `code/lean/Book/Ch25/`; we start with `Typestate.lean`.

```
/-- The states of the protocol, as an ordinary inductive type. -/
inductive ConnState where
  | closed | connected | authenticated
  deriving Repr, DecidableEq

/-- A connection in state `s`. The state appears only in the type: no field
mentions it, and Lean, unlike Rust, does not mind. -/
structure Conn (s : ConnState) where
  host : String
  sent : List String    -- what has gone over the wire, for the demo
  deriving Repr
```

The states are an ordinary inductive type, `ConnState`, with three values. That is already a difference from Rust, where the states had to be three separate types: in Lean, a type can be parameterized by a *value* of `ConnState`, so one type of states suffices. `Conn s` is a structure with a parameter `s`, and Lean does not mind that no field mentions it, so there is no `PhantomData`. Here is the type of `Conn` itself:

```
/-- info: Conn : ConnState → Type -/
#guard_msgs in
#check @Conn
```

`Conn` is a *function from states to types*: a family of types, one for each state. Such a function is called an **indexed family** of types. (Strictly speaking, `s` is a *parameter* of the structure; Chapter 30 explains the difference between parameters and indices, which does not matter here.) The operations are functions between members of the family:

```
def Conn.new (host : String) : Conn .closed := ⟨host, []⟩

def Conn.connect (c : Conn .closed) : Conn .connected :=
  ⟨c.host, c.sent ++ ["HELLO"]⟩

def Conn.login (c : Conn .connected) (user : String) : Conn .authenticated :=
  ⟨c.host, c.sent ++ [s!"LOGIN {user}"]⟩

def Conn.query (c : Conn .authenticated) (sql : String) : Conn .authenticated :=
  ⟨c.host, c.sent ++ [s!"QUERY {sql}"]⟩

/-- Available in every state: `s` is an implicit argument. -/
def Conn.close {s : ConnState} (c : Conn s) : Conn .closed :=
  ⟨c.host, c.sent ++ ["BYE"]⟩
```

The types are the transition table: `connect` maps `Conn .closed` to `Conn .connected`, and so on. `close` takes an implicit `{s : ConnState}`, so it works in every state; this is Lean's version of Rust's `impl<S: State> Connection<S>` (and we could restrict it to open states with an extra hypothesis, much as `Open` did). A complete session runs as expected:

```
/-- info: { host := "db", sent := ["HELLO", "LOGIN lab", "QUERY SELECT 1", "BYE"] } -/
#guard_msgs in
#eval ((Conn.new "db").connect.login "lab" |>.query "SELECT 1").close
```

(`e |>.query "SELECT 1"` is Lean's notation for `(e).query "SELECT 1"`: it calls `query` on the result of everything to its left, without extra parentheses.) And querying before logging in is a type error:

```
/--
error: Application type mismatch: The argument
  (Conn.new "db").connect
has type
  Conn ConnState.connected
but is expected to have type
  Conn ConnState.authenticated
in the application
  (Conn.new "db").connect.query
-/
#guard_msgs in
def tooEarly := (Conn.new "db").connect.query "SELECT 1"
```

The message has the same content as Rust's E0599, phrased as a mismatch between the type of the argument and the type `query` expects.

But Lean is not affine, and this is perfectly legal:

```
/-- Lean is not affine: the old state `c` can be used twice. For an
immutable value this is harmless; it would not be for a live socket. -/
def connectTwice (c : Conn .closed) : Conn .connected × Conn .connected :=
  (c.connect, c.connect)
```

The old state `c` is used twice. In Lean this is harmless: a `Conn .closed` is an immutable value, a *description* of a closed connection, not the connection itself. Both results are correct descriptions of what you would get by connecting it. Rust needs affinity for typestate because its values *are* the resources (a socket, a file, an instrument on a serial port), and two live views of one resource can disagree. Lean programs that talk to the outside world do it inside `IO`, where the order of effects is fixed by the monad (Chapter 20), and Lean values that stand for external resources, such as file handles, are not typestated by the standard library.

States that depend on run-time values: a vault

So far the states were a fixed, finite set. In Lean the index of a family can be any value, including one computed at run time. Here is a vault whose type records its balance:

```
/-- A vault whose type records its balance. The balance is not a field: it
is the index, and `Vault.balance` reads it back from the type. -/
structure Vault (balance : Nat) where
  owner : String

def Vault.open (owner : String) : Vault 0 := ⟨owner⟩

def Vault.deposit {b : Nat} (v : Vault b) (amount : Nat) : Vault (b + amount) :=
  ⟨v.owner⟩

/-- Withdrawing requires a proof that the balance suffices. The body does not
use the proof (hence the underscore); the type checker does. -/
def Vault.withdraw {b : Nat} (v : Vault b) (amount : Nat)
    (_enough : amount ≤ b) : Vault (b - amount) :=
  ⟨v.owner⟩

def Vault.balance {b : Nat} (_ : Vault b) : Nat := b
```

`Vault 100` is the type of vaults holding 100 units. The balance is not a field: it is the index, and `Vault.balance` reads it back from the type (the implicit argument `{b : Nat}` is a real argument at run

time, filled in by the type checker). `deposit` returns a vault whose type says the balance went up. `withdraw` takes an extra argument, a *proof* that the amount does not exceed the balance. That is a Lean feature with no counterpart in Rust: a function can demand evidence of a fact about its arguments (Chapter 1 met it with array indexing). The body ignores the proof (hence the name `_enough`); its only job is to make the call type-check.

```
/-- The type checker computes the balance: `0 + 100 - 30` is `70`. -/
def vault70 : Vault 70 :=
  ((Vault.open "lab").deposit 100).withdraw 30 (by decide)

/-- info: 70 -/
#guard_msgs in
#eval vault70.balance
```

The type checker does the bookkeeping: depositing 100 into a `Vault 0` gives a `Vault (0 + 100)`, and withdrawing 30 gives a `Vault (0 + 100 - 30)`, which Lean accepts as a `Vault 70` because the two indices compute to the same number. The proof obligation `30 ≤ 0 + 100` is discharged by `decide`. Overdrawing fails:

```
/--
error: Tactic `decide` proved that the proposition
  30 ≤ 0 + 10
is false
-/
#guard_msgs in
def overdrawn := ((Vault.open "lab").deposit 10).withdraw 30 (by decide)
```

This is a rejection of the same kind as Rust's E0599, a protocol violation found before the program runs, except that the "state" is a number and the rule is arithmetic.

When the amount is only known at run time, no one can write the proof in advance. The program must test the condition, and Lean's *dependent* `if` turns the test into a proof:

```
/-- An amount known only at run time: the dependent `if` tests the condition
and, in the `then` branch, hands us a proof `h` of it. -/
def tryWithdraw {b : Nat} (v : Vault b) (amount : Nat) :
    Option (Vault (b - amount)) :=
  if h : amount ≤ b then some (v.withdraw amount h) else none

/-- info: none -/
#guard_msgs in
#eval (tryWithdraw ((Vault.open "lab").deposit 10) 30).map Vault.balance

/-- info: some 20 -/
#guard_msgs in
#eval (tryWithdraw ((Vault.open "lab").deposit 50) 30).map Vault.balance
```

In `if h : amount ≤ b then ... else ...`, the name `h` is bound to a proof of `amount ≤ b` inside the `then` branch (and to a proof of its negation in the `else` branch). The run-time check and the static guarantee meet: the check produces exactly the evidence that `withdraw` demands. The result type `Option (Vault (b - amount))` is still precise about the balance in the success case.

If even the balance is only known at run time, for example after applying a list of transactions read from a file, we can no longer write it in the type. We package it with the vault instead:

```
/-- A vault with *some* balance: a dependent pair of the balance and a vault
of that balance. This is how a balance computed at run time is packaged. -/
abbrev AnyVault := (b : Nat) × Vault b

/-- Apply a list of transactions (positive: deposit, negative: withdrawal).
Withdrawals that would overdraw are skipped. -/
def applyAll : AnyVault → List Int → AnyVault
  | v, [] => v
  | ⟨b, v⟩, d :: ds =>
    if d ≥ 0 then
      applyAll ⟨b + d.toNat, v.deposit d.toNat⟩ ds
    else if h : d.natAbs ≤ b then
      applyAll ⟨b - d.natAbs, v.withdraw d.natAbs h⟩ ds
    else
      applyAll ⟨b, v⟩ ds

/-- info: 90 -/
#guard_msgs in
#eval (applyAll ⟨0, Vault.open "lab"⟩ [100, -30, -500, 20]).1
```

`(b : Nat) × Vault b` is a **dependent pair**: a number `b` together with a vault whose type mentions *that* `b` (Chapter 10 introduced these Σ-types). It is the Lean counterpart of an existential type (Chapter 15): "a vault with some balance". `applyAll` takes such a pair apart with the pattern `⟨b, v⟩`, and builds a new one after each transaction; withdrawals are guarded by a dependent `if`, so even inside this loop no withdrawal happens without a proof. Here, 100 is deposited, 30 withdrawn, the withdrawal of 500 is skipped, and 20 deposited: 90.

We can still prove things about the result:

```
/-- Whatever the transactions, the result is a well-formed vault: its type
index *is* its balance, and it is a natural number, so it never went
negative. What we can still prove is how deposits add up. -/
theorem applyAll_deposits (b : Nat) (v : Vault b) (ds : List Nat) :
    (applyAll ⟨b, v⟩ (ds.map Int.ofNat)).1 = b + ds.sum := by
  induction ds generalizing b with
  | nil => simp [applyAll]
  | cons d ds ih =>
    -- a deposit: the `d ≥ 0` branch is taken
    simp only [List.map_cons, applyAll, Int.ofNat_eq_natCast, Int.natCast_nonneg,
      ↓reduceIte, Int.toNat_natCast, List.sum_cons]
    rw [ih]
    omega
```

The statement says that applying a list of deposits (natural numbers, converted to integers by `Int.ofNat`) adds their sum to the balance. The proof is by induction on the list, generalizing the starting balance `b`. In the step case, `simp only` unfolds one step of `applyAll`, and the listed lemmas let it decide the `if`: a converted natural number is nonnegative (`Int.natCast_nonneg`), so the deposit branch is taken (`↓reduceIte` simplifies an `if` whose condition is now `True`), and converting back gives the original number (`Int.toNat_natCast`). Rewriting with the induction hypothesis leaves `b + d + sum ds = b + (d + sum ds)`, which `omega` proves.

Rust's closest tool is const generics (Chapter 30): a `Vault<const B: u64>` can record a balance known at compile time. But stable Rust cannot compute with const parameters in types (`Vault<{ B - A }>` requires an unstable feature), and a value known only at run time can never appear in a type. A Rust vault would check the balance at run time and return a `Result`, perhaps with a newtype whose constructor is the only place the check happens.

25.8 Protocols as inductive relations

The typestate API encodes the protocol *implicitly*, in the types of the operations. To reason about protocols, we also want them as explicit objects. The file `code/lean/Book/Ch25/Protocol.lean` does this for the lab instrument:

```
inductive Mode where
  | idle | calibrated | measuring
  deriving DecidableEq

inductive Action where
  | calibrate | start | sample | stop | powerDown
  deriving DecidableEq
```

```
/-- `Step m a m'`: in mode `m`, action `a` is allowed and leads to mode `m'`. -/
inductive Step : Mode → Action → Mode → Prop where
  | calibrate : Step .idle .calibrate .calibrated
  | start     : Step .calibrated .start .measuring
  | sample    : Step .measuring .sample .measuring
  | stop      : Step .measuring .stop .calibrated
  | powerDown : Step .calibrated .powerDown .idle

/-- `Run m tr m'`: the trace `tr` is allowed from `m` and ends in `m'`. -/
inductive Run : Mode → List Action → Mode → Prop where
  | nil  {m : Mode} : Run m [] m
  | cons {m₁ m₂ m₃ : Mode} {a : Action} {as : List Action} :
      Step m₁ a m₂ → Run m₂ as m₃ → Run m₁ (a :: as) m₃
```

`Step` is the transition relation δ, written as an inductive predicate in the style of the typing rules of Chapter 3: each constructor is an axiom, one per arrow of the diagram. `Run m tr m'` says that the trace `tr`, a list of actions, is allowed from mode `m` and ends in `m'`. It has two rules: the empty trace stays where it is; and a step followed by a run is a run.

A relation is good for proofs; for computation we want a function. `next` is the transition function, which returns `none` for a forbidden action, and `accepts` runs it along a trace:

```
/-- The transition function: the protocol as a run-time check. -/
def next : Mode → Action → Option Mode
  | .idle, .calibrate => some .calibrated
  | .calibrated, .start => some .measuring
  | .measuring, .sample => some .measuring
  | .measuring, .stop => some .calibrated
  | .calibrated, .powerDown => some .idle
  | _, _ => none

def accepts : Mode → List Action → Option Mode
  | m, [] => some m
  | m, a :: as => (next m a).bind fun m' => accepts m' as

/-- info: some idle -/
#guard_msgs in
#eval accepts .idle [.calibrate, .start, .sample, .sample, .stop, .powerDown]

/-- info: none -/
#guard_msgs in
#eval accepts .idle [.calibrate, .sample]
```

(`Option.bind` continues with the next action if the previous one was allowed, and propagates `none` otherwise, like `?` in Rust.) This is the run-time version of the protocol, what a `state` field and a check in each method would compute. The two versions agree, and proving it is our first theorem:

```
theorem next_iff {m m' : Mode} {a : Action} : next m a = some m' ↔ Step m a m' := by
  constructor
  · intro h
    -- 15 combinations of mode and action; `simp [next]` discards the ones
    -- where `next` is `none`, and each remaining one is a constructor.
    cases m <;> cases a <;> simp [next] at h <;> subst h <;> constructor
  · intro h
    cases h <;> rfl

theorem accepts_iff {m m' : Mode} {tr : List Action} :
    accepts m tr = some m' ↔ Run m tr m' := by
  induction tr generalizing m with
  | nil =>
    constructor
    · intro h
      simp only [accepts, Option.some.injEq] at h
      subst h
      exact .nil
    · intro h
      cases h
      rfl
  | cons a as ih =>
    constructor
    · intro h
      simp only [accepts, Option.bind_eq_some_iff] at h
      obtain ⟨m₂, h₁, h₂⟩ := h
      exact .cons (next_iff.mp h₁) (ih.mp h₂)
    · intro h
      cases h with
      | cons hs hr => simp [accepts, next_iff.mpr hs, ih.mpr hr]

/-- A concrete run, proved by computation through `accepts_iff`. -/
example : Run .idle [.calibrate, .start, .sample, .stop, .powerDown] .idle :=
  accepts_iff.mp rfl
```

`next_iff` says that the function and the relation agree on single steps. Left to right, we split on the mode and the action with `cases m <;> cases a`, which produces 15 goals (the combinator `<;>` applies the next tactic to every goal the previous one produced). In each, `simp [next] at h` computes `next`: where it is `none`, the hypothesis `none = some m'` is false and the goal closes; where it is `some`, `subst h` replaces `m'` by the computed mode, and `constructor` finds the `Step` constructor that proves it. Right to left, `cases h` considers each of the five constructors, and in each `next` computes to the right value, so `rfl` closes it.

`accepts_iff` lifts this to traces by induction on the trace. In the `nil` case, `accepts m []` is `some m`, which matches `Run.nil`. In the `cons` case, left to right: `Option.bind_eq_some_iff` says that a `bind` succeeded exactly when the first step gave some `m₂` (`h₁`) and the rest succeeded from there (`h₂`); then `next_iff` turns `h₁` into a `Step` and the induction hypothesis turns `h₂` into a `Run`. Right to left, we take the `Run` apart and rebuild the computation with `simp`. The final `example` shows the payoff: to prove a concrete run, we just let `accepts` compute (`rfl`), and `accepts_iff.mp` converts the computation into a proof.

Properties of traces

Now we can prove properties of *all* runs. The first one says that measurements are properly bracketed: along any run, the number of `start`s and `stop`s agree, up to a measurement that may be in progress at either end.

```
/-- 1 in the mode where a measurement is running, 0 elsewhere. -/
def running : Mode → Nat
  | .measuring => 1
  | _ => 0

/-- Along any run, starts and stops balance, up to the measurement that may
be running at the beginning or at the end. -/
theorem run_balance {m m' : Mode} {tr : List Action} (h : Run m tr m') :
    tr.count .start + running m = tr.count .stop + running m' := by
  induction h with
  | nil => simp
  | cons hs hr ih =>
    -- one case per kind of step; `simp_all` unfolds the counts, `omega`
    -- does the arithmetic
    cases hs <;> simp_all [running] <;> omega

/-- A complete session (from idle back to idle) stops every measurement it
starts. -/
theorem complete_balanced {tr : List Action} (h : Run .idle tr .idle) :
    tr.count .start = tr.count .stop := by
  simpa [running] using run_balance h
```

The function `running` is 1 in the measuring mode and 0 elsewhere. The statement of `run_balance` is the *generalized* form of what we want: it holds between any two modes, which is what makes the induction go through. The proof is by induction on the derivation of `Run` (Chapter 4 introduced induction on derivations): in the `nil` case both sides are the same; in the `cons` case, `cases hs` considers each kind of step, and for each one `simp_all` computes the counts of the extended trace from the induction hypothesis, and `omega` checks the arithmetic. For example, a `start` step from `calibrated` to `measuring` adds 1 to the `start` count on the left and moves `running` from 0 to 1 on the right. The corollary `complete_balanced` specializes it to runs from `idle` to `idle`, where `running` is 0 at both ends: a complete session stops every measurement it starts.

The second property: a nonempty run that ends in `idle` ends with `powerDown`, the only action that leads to `idle`.

```
/-- A nonempty run that ends in `idle` ends with `powerDown`: it is the only
action that leads to `idle`. -/
theorem ends_with_powerDown {m m' : Mode} {tr : List Action} (h : Run m tr m')
    (hm : m' = .idle) (hne : tr ≠ []) : tr.getLast? = some .powerDown := by
  induction h with
  | nil => contradiction
  | cons hs hr ih =>
    rename_i m₁ m₂ m₃ a as
    cases as with
    | nil =>
      -- the step `hs` is the last one, and it leads to `idle`
      cases hr
      subst hm
      cases hs
      rfl
    | cons b bs =>
      -- the last action is the last action of the rest of the run
      have := ih hm (by simp)
      simpa [List.getLast?_cons_cons] using this
```

The statement takes the final mode as a variable `m'` with a separate hypothesis `hm : m' = .idle`. That is a standard trick: `induction` on `h` needs the indices of `Run m tr m'` to be variables, and if we wrote `.idle` directly in the type of `h`, Lean would refuse (try it: the error says the index is not a variable). In the `cons` case, `rename_i` names the pieces of the step, and we look at the rest of the trace `as`.

If it is empty, the step was the last one: `cases hr` learns from `Run m₂ [] m₃` that `m₂ = m₃`, `subst hm` makes that mode `idle`, and `cases hs` finds that the only step into `idle` is `powerDown`. If the rest is not empty, the last action of the whole trace is the last action of the rest (`List.getLast?_cons_cons`), and the induction hypothesis gives it.

Typed programs follow the protocol

Finally, we connect the protocol back to typestate. In Rust, a sequence of calls like `probe.calibrate(..).start()` is accepted when each call is allowed in the state the previous one produced. We can describe such sequences as data, a family of types indexed by *two* modes, the start and the end:

```
/-- Programs that drive the instrument from mode `i` to mode `j`. Each
constructor is one operation of the typestate API: `calibrate` turns a
program that starts calibrated into one that starts idle. -/
inductive Prog : Mode → Mode → Type where
  | done {m : Mode} : Prog m m
  | calibrate {m : Mode} : Prog .calibrated m → Prog .idle m
  | start {m : Mode} : Prog .measuring m → Prog .calibrated m
  | sample {m : Mode} : Prog .measuring m → Prog .measuring m
  | stop {m : Mode} : Prog .calibrated m → Prog .measuring m
  | powerDown {m : Mode} : Prog .idle m → Prog .calibrated m

/-- The actions a program performs, in order. -/
def Prog.trace {i j : Mode} : Prog i j → List Action
  | .done => []
  | .calibrate p => .calibrate :: p.trace
  | .start p => .start :: p.trace
  | .sample p => .sample :: p.trace
  | .stop p => .stop :: p.trace
  | .powerDown p => .powerDown :: p.trace

def shift : Prog .idle .idle :=
  .calibrate <| .start <| .sample <| .sample <| .stop <| .powerDown .done

/-- info: [calibrate, start, sample, sample, stop, powerDown] -/
#guard_msgs in
#eval shift.trace
```

`Prog i j` is the type of programs that take the instrument from mode `i` to mode `j`. Each constructor adds one operation at the front: `calibrate` takes a program that starts in `calibrated` and returns one that starts in `idle`. This is the typestate API turned into a data type, with the same discipline: you cannot build a `Prog` that samples before calibrating, because the types do not fit. `trace` lists the actions of a program, and `shift` is one complete session.

The theorem says that the types guarantee protocol compliance:

```
/-- **Typed programs follow the protocol**: the trace of a `Prog i j` is a run
from `i` to `j`. -/
theorem Prog.trace_run {i j : Mode} (p : Prog i j) : Run i p.trace j := by
  induction p with
  | done => exact .nil
  | calibrate p ih => exact .cons .calibrate ih
  | start p ih => exact .cons .start ih
  | sample p ih => exact .cons .sample ih
  | stop p ih => exact .cons .stop ih
  | powerDown p ih => exact .cons .powerDown ih

/-- **Every complete session ends in the final state**, and so the run-time
check accepts its trace with result `idle`, its starts and stops balance, and
```

```
(if it does anything) it ends by powering down. -/
theorem session_complete (p : Prog .idle .idle) :
    accepts .idle p.trace = some .idle ∧
    p.trace.count .start = p.trace.count .stop ∧
    (p.trace ≠ [] → p.trace.getLast? = some .powerDown) :=
  ⟨accepts_iff.mpr p.trace_run,
   complete_balanced p.trace_run,
   ends_with_powerDown p.trace_run rfl⟩
```

`Prog.trace_run` is proved by induction on the program: each constructor corresponds to one `Step` constructor, and `Run.cons` adds it to the run of the rest. Then `session_complete` collects the consequences for a complete session, a `Prog .idle .idle`: the run-time check `accepts` agrees that the trace is allowed and ends in the final state `idle`, every measurement started is stopped, and a session that does anything ends by powering down. The proof is a triple of applications of the theorems above to `p.trace_run`; the anonymous constructor `⟨_, _, _⟩` builds the conjunction.

Compare this with Rust. The function signature `fn shift(i: Instrument<Idle>) -> Instrument<Idle>` *looks* like it demands a complete session, but it does not: the body may drop the measuring instrument half-way and return a brand-new idle one made with `Instrument::new`. A `Prog` has no constructor for "abandon" or "start over", so the type `Prog .idle .idle` really does describe only complete sessions.

Session types in Lean

The same idea gives session types their full strength. In `code/lean/Book/Ch25/Session.lean`, sessions are an inductive type and duality is a function:

```
/-- The types of the messages. -/
inductive Ty where
  | nat | bool
  deriving DecidableEq

/-- The Lean type of a message of type `t`. -/
abbrev Ty.denote : Ty → Type
  | .nat => Nat
  | .bool => Bool

inductive Session where
  | send (t : Ty) (k : Session)   -- send a `t`, continue as `k`
  | recv (t : Ty) (k : Session)   -- receive a `t`, continue as `k`
  | done
  deriving DecidableEq

def Session.dual : Session → Session
  | .send t k => .recv t k.dual
  | .recv t k => .send t k.dual
  | .done => .done
```

Message types are a small inductive `Ty` with a function `Ty.denote` into Lean types, the same technique as the intrinsically typed evaluator of Chapter 8. Duality is an involution, by induction on the session:

```
/-- Duality is an involution: the other side of the other side is you. -/
theorem Session.dual_dual (s : Session) : s.dual.dual = s := by
  induction s with
  | send t k ih => simp [dual, ih]
  | recv t k ih => simp [dual, ih]
  | done => rfl
```

In each `send` and `recv` case, `simp [dual, ih]` unfolds `dual` twice and uses the induction hypothesis for the continuation; `done` is its own dual by definition. Now a process is indexed by the session it follows, and two processes with dual sessions can be run against each other:

```
/-- A process that follows session `s`. Sending needs a value of the right
type; receiving needs a continuation for every value that may arrive. -/
inductive Proc : Session → Type where
  | send {t : Ty} {k : Session} : t.denote → Proc k → Proc (.send t k)
  | recv {t : Ty} {k : Session} : (t.denote → Proc k) → Proc (.recv t k)
  | done : Proc .done

def Ty.show : (t : Ty) → t.denote → String
  | .nat, n => toString n
  | .bool, b => toString b

/-- Run two processes with dual sessions against each other, recording the
messages. There are only three cases: the types rule out every mismatch
(both sending, both receiving, one finished early), and the recursion is
structural, so the conversation always completes. -/
def talk : {s : Session} → Proc s → Proc s.dual → List String
  | .send t _, .send v p, .recv f => s!"→ {t.show v}" :: talk p (f v)
  | .recv t _, .recv f, .send v q => s!"← {t.show v}" :: talk (f v) q
  | .done, .done, .done => []
```

The function `talk` is the most interesting definition of the chapter, because of what it *lacks*. It matches the two processes simultaneously, and there are only three cases. If the first process is sending, its session is `.send t k`, so the second process has type `Proc (.send t k).dual`, which Lean computes to `Proc (.recv t k.dual)`: the second process *must* be receiving, a value of the same type `t`. Lean checks that every combination not listed (both sending, both receiving, one finished while the other is not) is impossible because the types cannot match, so no case is missing. And the recursion is structural, so `talk` terminates. A total function from two dual processes to a transcript is a proof that dual processes always communicate to completion, with no mismatched message and no deadlock.

```
/-- The client's protocol: send a sensor number, receive whether it is
in service. -/
def Query : Session := .send .nat (.recv .bool .done)

def client : Proc Query :=
  .send (7 : Nat) (.recv fun _inService => .done)

/-- The server follows the dual protocol: receive, then send. -/
def server : Proc Query.dual :=
  .recv fun (sensor : Nat) => .send (decide (sensor < 10)) .done

/-- info: ["→ 7", "← true"] -/
#guard_msgs in
#eval talk client server
```

The server is written against `Query.dual`, and its `recv` receives a `Nat` because that is what the dual session says. This is the guarantee the Rust encoding could not give: Lean's totality ensures that the protocol finishes, which is the "at least once" half that affinity does not provide.

25.9 Putting it together: the lab instrument, end to end

Let us line up everything this chapter did with one protocol, the lab instrument.

Concept	Rust	Lean
state	a type: `Idle`, `Calibrated`, `Measuring`	a value of `Mode`
data known in a state	fields of the state type	(not modeled)
allowed transition	a method in an `impl Instrument<S>` block	a constructor of `Step`
wrong-state call	E0599, no method found	no constructor applies
old state reused	E0382, it was moved	allowed: values are immutable
a program	a chain of method calls	a value of `Prog i j`
run-time check	a destructor that complains	`accepts`
complete session	not enforceable	`Prog .idle .idle`, proved complete

Read the rows from top to bottom and the story of the chapter appears. The protocol's diagram becomes types in both languages. In Rust, the types reject operations in the wrong state (E0599), and affinity rejects the reuse of a state that was consumed (E0382), which matters because a Rust value is the resource itself. Rust cannot make a measurement finish: a value may always be dropped. What it offers instead is a run-time complaint, which needs care to write (E0366 and E0509 shape the code), and the closure-based design that keeps the protocol under the library's control. In Lean, the same diagram becomes an inductive relation; typed programs are a data type whose values provably follow the relation; complete sessions provably end in the final state with every measurement stopped; and the runtime checker provably agrees with the relation.

The comparison also says something about the question of this book. Typestate is one of the most practical consequences of Rust's ownership discipline, and it works because ownership is affine: "at most once" is exactly what makes a consumed state unusable. The guarantee it misses, "at least once", is exactly the difference between affine and linear. Chapter 33 collects this observation with the others.

25.10 Common confusions

"**Typestate needs `PhantomData`.**" Only when the state carries no data. The builder and the instrument use their type parameters as the types of real fields, and need no marker. `PhantomData` is for the case where the state is purely a label, as in the connection.

"**With typestate, protocol errors are impossible.**" Calling an operation in the wrong state is impossible. Abandoning the protocol half-way is not, and when a transition depends on run-time data (a login that may fail), the outcome comes back as a `Result` or an enum of states that must be handled at run time.

"**A sealed trait is a security feature.**" It is an API design tool. It prevents other crates from implementing the trait, so that the set of states stays closed and the library can rely on knowing all of them. It has nothing to do with access control at run time.

"**`#[must_use]` on a typestate type makes the protocol linear.**" It warns when a transition's result is discarded on the spot, which catches a common slip. It does nothing about a value that is bound, kept, and dropped later in the wrong state.

"**Lean's typestate is weaker because it is not affine.**" It is different. Lean values are immutable, so reusing an old state is not a lie, and Lean's types can mention run-time values (the vault) and its totality can guarantee completion (the session types), which Rust cannot express.

25.11 Summary

- Typestate turns a protocol's states into types and its transitions into functions between them; calling an operation in the wrong state becomes a type error (E0599 in Rust), checked before the program runs.
- In Rust, transitions take `self` by value. Affinity makes the old state unusable (E0382); without it, two views of one resource could disagree.
- The toolkit: empty enums or unit structs as state labels, `PhantomData` when the state is only a label (E0392 otherwise), sealed traits to close the set of states (E0277 for outsiders), trait bounds for groups of states, states that carry the data known in them, `Result` for fallible transitions.
- Builders with required fields use one type parameter per field; generic `impl` blocks allow any order, and `build` exists only when everything is set.
- Session types describe conversations; duality (! and ? swapped) is an involution, and dual endpoints match message for message. A Rust encoding guarantees no wrong message, but not that the conversation finishes.
- Affinity cannot force completion. `#[must_use]` warns about ignored results; a complaining destructor reports abandonment at run time, but must live on a type without parameters or be generic (E0366) and must be disarmed by the normal transitions (E0509); closure-based APIs keep the protocol under control.
- Lean expresses typestate with indexed families (`Conn : ConnState → Type`), lets states depend on run-time values (a vault indexed by its balance, with a proof obligation for withdrawals and dependent `if` and pairs for run-time amounts), and describes protocols as inductive relations.
- We proved that a run-time checker agrees with the protocol relation, that runs from `idle` to `idle` stop every measurement and end with `powerDown`, that typed programs follow the protocol, and that dual Lean processes always talk to completion.

25.12 Exercises

1. Design a `ServerBuilder` whose `build()` method exists only after both `name` and `port` have been set, in either order, with an optional `max_connections` available in every state. How many type parameters does it need, and how many distinct builder types can occur? *Hint:* follow `RequestBuilder`: one parameter per required field, with a "missing" marker and a "present" type for each; four types in all.
2. Add a `Maintenance` mode to the instrument, reachable from `Idle` by `service` and returning to `Idle` by `release`. Update the Rust module, the Lean `Mode`, `Action`, `Step`, `next` and `Prog`, and check which proofs still go through unchanged. *Hint:* `next_iff` and `accepts_iff` need no new

ideas, but the automation in `next_iff` now has more cases; `ends_with_powerDown` is now *false*. Why? State and prove the corrected version.

3. Prove in `Protocol.lean` that every complete run calibrates as often as it powers down: `Run .idle tr .idle → tr.count .calibrate = tr.count .powerDown`. *Hint:* generalize as in `run_balance`, with a function that is 0 in `idle` and 1 in the two other modes.
4. Write the closure-based `measure` method sketched in "Keeping control" for `ch25_typestate.rs`, with a `Recorder` type that only allows recording samples. Then explain why a caller can no longer abandon a measurement, and the one way in which the measurement can still be interrupted. *Hint:* `Recorder<'a>(&'a mut Instrument<Measuring>)` with a single `record` method; the interruption is a panic inside the closure.
5. Extend `Session.lean` with *choice*: a constructor `offer (k₁ k₂ : Session)` (the other side chooses a branch) and `choose (k₁ k₂ : Session)` (this side chooses), dual to each other. Extend `dual`, `dual_dual`, `Proc` and `talk`. *Hint:* a `Proc (.choose k₁ k₂)` is a choice of one branch with a process for it (a `Bool` and a dependent `if`, or two constructors); a `Proc (.offer k₁ k₂)` needs a process for each branch.
6. In `Typestate.lean`, prove that `tryWithdraw v amount` returns `none` exactly when `b < amount`. *Hint:* unfold `tryWithdraw` and use `split` (or `by_cases h : amount ≤ b`) to follow the dependent `if`; `simp` and `omega` handle the rest.